

1. AR(1) Model - Estimation and Forecast.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# Dataset
X = np.array([2.0, 2.8, 3.3, 3.7, 4.1, 4.5])

# Create lagged variable
X_lag = X[:-1]
X_current = X[1:]

# Add constant for OLS
X_lag_const = sm.add_constant(X_lag)

# Fit AR(1) via OLS
model = sm.OLS(X_current, X_lag_const).fit()
print(model.summary())

# Extract phi
phi = model.params[1]
print("Estimated phi:", phi)

# Forecast X7
X6 = X[-1]
forecast_X7 = phi * X6
print("Forecast X7:", forecast_X7)
```

2. MA(1) - ACF & Method of Moments

```
import numpy as np
from statsmodels.tsa.stattools import acf

# Dataset
X = np.array([1.2, -0.4, 0.9, -1.1, 0.3, 0.5])

# Basic stats
print("Mean:", np.mean(X))
print("Variance:", np.var(X, ddof=1))

# ACF (lags 0,1,2)
acf_vals = acf(X, nlags=2)
print("ACF:", acf_vals)

rho1 = acf_vals[1]

# Method of moments for MA(1):
#  $\rho(1) = \theta / (1 + \theta^2)$ 
# Solve approximately

import sympy as sp

theta = sp.symbols('theta')
solution = sp.solve(rho1 - theta/(1 + theta**2), theta)
print("Theta solutions:", solution)
```

3. ARMA(1,1). Model Fit.

```
import numpy as np
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA

# Dataset
X = np.array([0.5, 1.1, 0.9, 1.4, 1.2, 1.5])

# Fit ARMA(1,1) (ARIMA with d=0)
model = ARIMA(X, order=(1, 0, 1))
results = model.fit()

print(results.summary())
```

4. ACF & PACF Plot (Model Identification)

```
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import
plot_acf, plot_pacf

X = np.array([2.0, 2.8, 3.3, 3.7, 4.1, 4.5])

plt.figure()
plot_acf(X, lags=4)
plt.show()

plt.figure()
plot_pacf(X, lags=4)
plt.show()
```

5. Forecasting (Manual + Model-Based)

```
import numpy as np

X = np.array([10, 12, 13, 15, 16])
phi = 0.7

# One-step ahead forecast
X6 = phi * X[-1]
print("Forecast X6:", X6)

# Two-step ahead forecast
X7 = phi * X6
print("Forecast X7:", X7)
```